

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Управление доступом с помощью списков контроля доступа.
Часть 2. Управление доступом к приложениям и системам баз данных»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: изучить механизмы контроля доступа, предоставляемые серверными приложениями и системами управления базами данных, и получить практические навыки настройки политики безопасности (администратор – пользователь – гость) для СУБД и веб-приложений.

Задание:

- а) Создать программу, которая позволяет установить веб-приложение или систему управления базами данных (СУБД).
- б) Создать программу, которая позволяет настроить механизм контроля доступа для выбранного веб-приложения или СУБД.
- в) Создать программу для проверки корректности работы настроенной политики безопасности выбранной СУБД или веб-приложения.
- г) Создать программу, которая позволяет удалить веб-приложение или СУБД, а также созданные ими объекты файловой системы.
- д) Оформить отчёт о проделанной работе.

Политика безопасности должна быть настроена следующим образом: в системе должны присутствовать администратор системы с полным доступом ко всем подсистемам и максимальными привилегиями, пользователь с полными либо частичными правами в подсистеме, предоставленной администратором, и гость с правами только на чтение отдельных фрагментов, предоставленных администратором.

Задание выполнено для документоориентированной СУБД MongoDB. В виде отдельного консольного C#-проекта (**Lab1.2-MongoDB**) реализован сценарий, последовательно выполняющий все четыре программы из задания как этапы одного сценария: установка, настройка контроля доступа, проверка и удаление. Все действия – реальные команды Linux (**docker**, **mongosh** и т. п.), запускаемые как внешние процессы; .NET здесь выступает только оркестратором сценария, а не исполнителем самой политики безопасности.

Краткое описание MongoDB

MongoDB – документоориентированная СУБД: данные хранятся не в таблицах со строками, а в коллекциях JSON-подобных документов (BSON). В лабораторной работе используется официальный Docker-образ **mongo:8.2-noble**. По умолчанию свежий экземпляр MongoDB принимает любые подключения без аутентификации – контроль доступа необходимо включить явно ключом **-auth** и настроить отдельно от установки. Модель прав в MongoDB строится вокруг ролей: встроенные роли (например, **root**, **readWrite**) назначаются пользователю в контексте конкретной базы данных, а для более точных ограничений можно определить собственную роль с перечнем разрешённых действий над конкретной коллекцией – этим

MongoDB напоминает списки контроля доступа из первой части лабораторной работы, только уровнем выше: не файл и не каталог, а документ и коллекция.

Реализация: MongoDB

Проект `Lab1.2-MongoDB` организован так же, как и программа из первой части лабораторной работы: `Program.cs` последовательно вызывает четыре этапа из каталога `Steps` (`Install`, `Configure`, `Verify`, `Cleanup`), а вспомогательный класс `Report` запускает внешние процессы и печатает результат каждой команды в формате `[OK]/[FAIL]`, а результат проверки прав доступа – в формате `[ALLOWED]/[DENIED]`.

1. Установка

Установка выполняется командами `docker pull mongo:8.2-noble` и `docker run`, с монтированием именованного тома `mongo_lab_data` для постоянного хранения данных. Контейнер намеренно запускается без `-auth`: MongoDB разрешает создать самого первого пользователя без пароля только через т. н. `localhost exception`, то есть до включения обязательной аутентификации.

2. Настройка контроля доступа

Пока аутентификация ещё не включена, программа создаёт администратора `admin` с встроенной ролью `root`, после чего контейнер перезапускается с ключом `-auth` (том с данными переиспользуется, поэтому созданный пользователь сохраняется). Далее от имени администратора создаются: пользователь `labuser` с ролью `readWrite` на базе `labdb` (полные права в предоставленной администратором «подсистеме» – одной прикладной базе данных) и пользователь `labguest`, которому назначена не встроенная, а специально созданная роль `guestReader`, разрешающая только действие `find` и только над одной коллекцией `public_notes` (см. листинг 1). Соседняя коллекция `private_notes` для гостя недоступна вовсе.

```

1 Report.Step(8, $"Create '{Config.GuestUser}' - read-only on
  ↳ '{Config.AppDb}.{Config.PublicCollection}' only");
2 MongoEvalAuth($"Create role '{Config.GuestRole}' (find on
  ↳ {Config.AppDb}.{Config.PublicCollection} only)",
3   $"db.getSiblingDB('{Config.AppDb}').createRole({{role:'{Config.GuestRol}
  ↳ e}','",
  ↳ +
4   $"privileges:[{{resource:{{db:'{Config.AppDb}',collection:'{Config.Publ}
  ↳ icCollection}}}},actions:['find']}}],roles:[]}})");
5 MongoEvalAuth($"Create user '{Config.GuestUser}' (role {Config.GuestRole})",
6   $"db.getSiblingDB('{Config.AppDb}').createUser({{user:'{Config.GuestUse}
  ↳ r}','pwd:'{Config.GuestPassword}',roles:[{{role:'{Config.GuestRole}'
  ↳ ,db:'{Config.AppDb}'}}]}})");

```

Листинг 1 – Создание гостевой роли и пользователя с правом чтения одной коллекции

3. Проверка корректности

Проверка выполняется той же утилитой `mongosh`, но уже от имени каждой из трёх учётных записей, и интерпретирует нулевой код завершения команды как **ALLOWED**, а ненулевой (сервер отклонил операцию как неавторизованную) – как **DENIED**. Полный протокол реального запуска программы приведён ниже (см. листинг 2). Администратор проходит любую проверку; `labuser` может читать и писать в `labdb`, но не может ни создать нового пользователя, ни обратиться к посторонней базе `otherdb`; `labguest` может прочитать только `public_notes` – чтение `private_notes` и любая запись отклоняются. Отдельно проверено, что клиент без учётных данных не может выполнить вообще ни одной операции – `-auth` действительно принудительно применяется сервером, а не является лишь опцией по умолчанию.

```

1  == Step 1: Pull the MongoDB server image ==
2  [OK] Pull MongoDB image
3  $ docker pull mongo:8.2-noble
4  == Step 2: Create a persistent data volume ==
5  [OK] Create data volume
6  $ docker volume create mongo_lab_data
7  == Step 3: Start MongoDB (bootstrap mode, no authentication yet) ==
8  [OK] Run MongoDB container
9  $ docker run -d --name mongo_lab -p 27017:27017 -v mongo_lab_data:/data/db mongo:8.2-noble
10 [OK] MongoDB accepted a connection and is ready
11 == Step 4: Create the administrator account through the localhost exception ==
12 [OK] Create admin user (role 'root' on 'admin')
13 $ docker exec mongo_lab mongosh --quiet --eval
   > db.getSiblingDB('admin').createUser({user:'admin',pwd:'AdminP@ss1',roles:[{role:'root',db:'admin'}]})
14 == Step 5: Restart MongoDB with authorization enforced ==
15 [OK] Stop the bootstrap container
16 $ docker rm -f mongo_lab
17 [OK] Run MongoDB container with --auth
18 $ docker run -d --name mongo_lab -p 27017:27017 -v mongo_lab_data:/data/db mongo:8.2-noble --auth
19 [OK] MongoDB is back up with authorization enforced
20 == Step 6: Seed the application database with public and private data ==
21 [OK] Insert a document into public_notes
22 $ docker exec mongo_lab mongosh -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
   > db.getSiblingDB('labdb').public_notes.insertOne({title:'public seed', visibility:'public'})
23 [OK] Insert a document into private_notes
24 $ docker exec mongo_lab mongosh -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
   > db.getSiblingDB('labdb').private_notes.insertOne({title:'private seed', visibility:'private'})
25 == Step 7: Create 'labuser' - readWrite on 'labdb' only ==
26 [OK] Create user 'labuser' (role readWrite on labdb)
27 $ docker exec mongo_lab mongosh -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
   > db.getSiblingDB('labdb').createUser({user:'labuser',pwd:'UserP@ss1',roles:[{role:'readWrite',db:'labdb'}]})
28 == Step 8: Create 'labguest' - read-only on 'labdb.public_notes' only ==
29 [OK] Create role 'guestReader' (find on labdb.public_notes only)
30 $ docker exec mongo_lab mongosh -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval db.getSiblingDB('labdb').createRole({ro
   > le:'guestReader',privileges:[{resource:{db:'labdb',collection:'public_notes'},actions:['find']}],roles:[]})
31 [OK] Create user 'labguest' (role guestReader)
32 $ docker exec mongo_lab mongosh -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
   > db.getSiblingDB('labdb').createUser({user:'labguest',pwd:'GuestP@ss1',roles:[{role:'guestReader',db:'labdb'}]})
33 == Step 9: Start Mongo Express so the setup can be checked visually ==
34 [OK] Pull Mongo Express image
35 $ docker pull mongo-express:1
36 [OK] Run Mongo Express container
37 $ docker run -d --name mongo_lab_express -p 8081:8081 --link mongo_lab:mongo_lab -e ME_CONFIG_MONGODB_SERVER=mongo_lab -e
   > ME_CONFIG_MONGODB_PORT=27017 -e ME_CONFIG_MONGODB_ADMINUSERNAME=admin -e ME_CONFIG_MONGODB_ADMINPASSWORD=AdminP@ss1 -e
   > ME_CONFIG_MONGODB_AUTH_DATABASE=admin -e ME_CONFIG_BASICAUTH=false mongo-express:1
38 [OK] Open http://localhost:8081 to browse 'labdb' (public_notes/private_notes) visually, connected as 'admin'
39 == Step 10: Verify the administrator's privileges ==
40 [ALLOWED] admin ListDatabases admin (cluster-wide)
41 == Step 11: Verify 'labuser' has full rights inside 'labdb' only ==
42 [ALLOWED] labuser Read labdb.public_notes
43 [ALLOWED] labuser Write labdb.private_notes
44 [DENIED ] labuser CreateUser labdb (admin-only op)
45 [DENIED ] labuser Read otherdb.secrets (no role granted)
46 == Step 12: Verify 'labguest' can only read 'public_notes' ==
47 [ALLOWED] labguest Read labdb.public_notes
48 [DENIED ] labguest Read labdb.private_notes
49 [DENIED ] labguest Write labdb.public_notes
50 == Step 13: Verify that an unauthenticated client is rejected outright ==
51 [DENIED ] (none) Write labdb.public_notes (no credentials)
52 == Step 14: Remove MongoDB, Mongo Express, the data volume and their images ==
53 [OK] Remove the container
54 $ docker rm -f mongo_lab
55 [OK] Remove the Mongo Express container
56 $ docker rm -f mongo_lab_express
57 [OK] Remove the data volume
58 $ docker volume rm mongo_lab_data
59 === Summary ===
60 Operations: 19 total, 0 failed
61 CreateUser allowed 0 / 1 denied 1
62 ListDatabases allowed 1 / 1 denied 0
63 Read allowed 2 / 4 denied 2
64 Write allowed 1 / 3 denied 2

```

Листинг 2 – Реальный протокол работы программы Lab1.2-MongoDB

4. Удаление

Завершающий этап удаляет оба контейнера (MongoDB и Mongo Express) и именованный том с данными. Удаление самих образов (mongo:8.2-noble и mongo-express:1) намеренно закомментировано в коде – команда присутствует, но не выполняется по умолчанию, – чтобы при повторных запусках демонстрации образы оставались заэкшированными и не требовали повторной загрузки; при необходимости полной очистки соответствующие строки достаточно раскомментировать.

Выводы

В ходе выполнения лабораторной работы был освоен механизм контроля доступа на уровне приложений и систем управления базами данных – в отличие от первой части, где права ограничивались правами файловой системы, здесь для MongoDB реализована и экспериментально подтверждена трёхуровневая политика безопасности «администратор – пользователь – гость», выраженная собственными, специфичными для этой СУБД средствами: встроенными и пользовательскими ролями с привилегиями на уровне базы данных и коллекции. Практически показано, что в основе этого механизма лежит принцип наименьших привилегий: пользователю и гостю явно и по отдельности выдаётся ровно тот объём прав, который необходим для их роли, а не урезается из более широкого набора.

Семантически лабораторная работа подтвердила, что задача администратора не сводится к однократной установке ПО: то, что MongoDB по умолчанию разрешает подключение вовсе без аутентификации, означает, что первым шагом всегда является явное включение и настройка контроля доступа – то же наблюдение, что и в первой части работы, только теперь на уровне не операционной системы, а конкретного приложения. Использование Docker-контейнера для установки также оказалось практичным решением: оно позволило безопасно выполнять реальные, а не имитационные команды установки и настройки, и гарантировало полное и воспроизводимое удаление всех созданных объектов на завершающем этапе.